Muhammad Adil Saeed.
24IC-0705 BCS-4J.

Assignment #02.

Homework.

4.1 1. Web Server Handling Multiple Requests.

Scenario
A web server recieves many user requests at same time.

Single-threaded approach:
- Handles one request at a time.
- Other users must wait until the current request finish

Multi-threaded
- Each request is handled in a seperate tread.
- Multiple users are served simultaneasly.

Performance improves as it reduces waiting time for users with better cpu utilization and with faster response time

2. File Download Management.

Downloading a larger file from internet.

Single-threaded approach.
- Downloads the file in one continous stream.
- Slower due to network limitation.

Multithreaded approach.
- Split file in chunks.
- Download each chunk using seperate threads.

It improve performeces as it utilizes full network bandwidth, with parallel downloads reduce total download time.

3. Image Filtering

Applying filters.

Single-threaded approach:
- Processes pixels one by one.
- Edit longer of larger image.

Multithread approach.
- Divides the image into sections
- Each thread processes a portion

Parellel computation speeds up processing with effficient use of multi-core CPUs, along with significant time reduction improve performances.

4.2. a) S = Speedup $\leq \frac{1}{S + \left(\frac{1-S}{N}\right)}$

$$\frac{1}{40 + \frac{60}{2}} = 1.4.3$$

b) $N = 4$.

$$S = \frac{1}{.4 + \frac{0.6}{4}} = 1.82.$$

4.3 The multithreaded web server described in Section 4.1 exhibits task parallelism, because each thread handles a seperate client request independently rather than dividing the same data among threads

4.4. User-level threads are managed in user-space by thread library, while kernel-level threads are manged by the OS. A key difference is if one user-level thread blocks, the entire process blocks, where as in kernel-thread, other threads can continue running even if one is block. User level is faster while kernel level are heavy application and slower

4.5 During a context switch between kernel-level threads, the kernel first saves the current thread's state into its Thread Control Block then it changes the thread state to ready or blocked. Next, the Scheduler selects another thread from the ready queue. Finally the kernel loads the saved content of the new thread and transfer.

4.6 When a thread is created, it uses light weight resources such as a program counter, register, stack and thread control block, while sharing code, data and files with other threads in the same process. In contrast when a process is created, it requires its own seperate memory space code, data, stack head and PCB. along with system resources. Thread creation is therefore faster

4.7 Yes, it is necessary to bind a real-time thread to an LWP in a many-to-many model. This is because real-time threads require guaranteed CPU access and predictabe execution.

4.8: 1. Calculation sum of a small array using multiple threads.
2. Sequential file processing (read → process → write) when each step depends on the previous one.

4.9 Multithreaded solutions perform better on a single processor when the program is I/O-bound or has frequent blocking operations while one-threaded waits for I/O e.g. Multithreading does not improve performance due to overhead.

4.10. b) Heap memory
c) Global variables

4.11 No, User-level threads cannot run in parallel on multiple processors because kernel threads runs as a single process, So there is no true parallelism and performance is similar to Single-processor system.

4.12. No, Using threads would not give the same benefits because threads share memory. A crash or bug in one tab could affect all tabs could affect all tabs. Seperate processes provide better stability and security

4.13. Yes, concurrency means multiple tasks make progress by sharing CPU time, but only one runs at a time on a single-core system. Parallelism means tasks run at the same time on multiple cores. So concurrency can exist without parallelism.

4.14. $S = \frac{1}{1-P+P/N}$. ∴ P = parallel.

1. 40% = P = 0.4.
   a) $\frac{1}{0.6 + 0.4/8} = 1.64$.
   b) $\frac{1}{0.6+0.4/16} = 1.60$.

2. .67
   a) $\frac{1}{0.33 + \frac{0.67}{2}} = 1.50$.
   b) $\frac{1}{0.33+\frac{0.67}{4}} = 2.01$.

3. .90
   a) $\frac{1}{0.1+\frac{0.9}{4}} = 3.08$.
   b) $\frac{1}{0.1+\frac{0.9}{8}} = 4.71$.

4.15
- Task parallelism
- Data parallelism
- Task parallelism
- Data parallelism
- Task parallelism.

4.16.

- Input/Output:
  Use 1 thread for input and 1 thread for output. Since I/O happens only once at the start and end with a single file, additional threads won't improve performance.
- CPU-intensive part:
  Use 4 threads. The system has 4 cores, and with one-to-one mapping, each thread can run on a seperate core, giving maximum parallelism.

4.17

a) First fork() = 2 processes.
Inside child, another fork() = 3 processes.
Final fork() = 3x2 = 6 processes.
Total processes = 6.

b) Number of threads
thread_crete() is executed by only the child after 2nd fork
2 processes execute it, each creates 1 thread.
Total threads created = 2.

4.18 Linux treats processes and threads the same using task structure with clone(), providing a flexible and unified model. In contrast, like windows use separate structures where a process contains and manages multiple threads. Thus, Linux uses a uniform approach, while others maintains a clear distinction between processes and threads.

4.19.
Lin C: Child: value = 5.
Line P: Parent: value = 0.

4.22.

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

int *numbers;
int n;
double avg;
int min_val, max_val;

void *average(void *arg){
    int sum = 0;
    for (int i=0; i<n; i++)
        sum += numbers[i];
    avg = (double)sum/n;
    pthread_exit(0);
}

void *minimum(void *arg){
    min_val = numbers[0];
    for (int i=1; i<n; i++)
        if (numbers[i] < min_val)
            min_val = numbers[i];
    pthread_exit(0);
}

void *maximum(void *arg){
    max_val = numbers[0];
    for (int i=1; i<n; i++)
        if (numbers[i] > max_val)
            max_val = numbers[i];
    pthread_exit(0);
}

int main(int argc, char *argv[]){
    if (argc<2){
        printf("Usage
    n = argc-1;
    for (int i=0; i<n; i++)
        numbers[i]
    pthread_t t1,t2,t3;

    pthread_create(&t1,NULL,average,NULL);
    pthread_create(&t2,NULL,minimum,NULL);
    pthread_create(&t3,NULL,maximum,NULL);

    pthread_join(t1,NULL);
    pthread_join(t2,NULL);
    pthread_join(t3,NULL);
    pthr
    printf("Average: %.of\n", avg);
    printf("Minimum %d\n", min_val);
    printf("Maximum %d\n", max_val);

    return 0;
}
```

./a.out 90 81 78 95 79
72 85

4.23

```
#include <stdio.h>
#include <stdlib.h>
# include <pthread>

int n;

int is_prime(int num){
    if (num<2)
        return 0;
    for (int i=2;i*i <=num ;i++){
        if (num %i ==0)
            return 0;
        }
        return 1;
    }
void *print_primes(void *arg){
     for (int i=2 ; i<=n ; i++){
        if(is_prime(i))
            printf("%d ",i);
     }
    printf("\n");
    pthread_exit(0);
}

int main(){
    int argc, char *argv[]){
      if (argc < 2){
        printf("Usage: %s\n", argv[0] );
        return 1;
      }
     n = atoi(argv[1]);
    pthread_t tid;
    pthread_create(&tid,NULL ,prime_prives, NULL);
    pthread_join (tid,NULL);
    return 0;
}
```

./a.out 20.

Name : Muhammad Adil Saeed

Roll No:24k-0705

QS2.

```
system@system:~/Desktop/Assignment_2$ gcc question2.c -o question2 -lpthread -lm
system@system:~/Desktop/Assignment_2$ ./question2
Max: 48.750000
Min: -18.000000
Hotspots: 11952
Coldspots: 1571
```

CODE:

```
#include <stdio.h>

#include <pthread.h>

#include <math.h>

#include <stdlib.h>


#define N 1000

#define TILE 100

#define MAX_THREADS 8

#define T_HOT 35

#define T_COLD -10


double globalMatrix[N][N];

double satelliteData[5][N][N];


double global_min = 1000000000;

double global_max = -1000000000;
```

```
double global_sum = 0;

int hotspot_count = 0;
int coldspot_count = 0;
int anomaly_count = 0;

pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

void initializeData() {
    for (int k = 0; k < 5; k++) {
        for (int i = 0; i < N; i++) {
            for (int j = 0; j < N; j++) {
                satelliteData[k][i][j] = (rand() % 71) - 20;
                if (rand() % 20 == 0) {
                    satelliteData[k][i][j] = NAN;
                }
            }
        }
    }
}

void* processSatellite(void* arg) {
    int id = *(int*)arg;
```

```
    for (int i = 1; i < N-1; i++) {
        for (int j = 1; j < N-1; j++) {
            if (isnan(satelliteData[id][i][j])) {
                satelliteData[id][i][j] =
                    (satelliteData[id][i-1][j] +
                     satelliteData[id][i+1][j] +
                     satelliteData[id][i][j-1] +
                     satelliteData[id][i][j+1]) / 4.0;
            }
        }
    }
    pthread_exit(NULL);
}

void mergeMatrices(int M) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            double sum = 0;
            int count = 0;
            for (int k = 0; k < M; k++) {
                if (!isnan(satelliteData[k][i][j])) {
                    sum += satelliteData[k][i][j];
                    count++;
```

```
            }
        }
        if (count > 0)
            globalMatrix[i][j] = sum / count;
    }
  }
}

int nextTile = 0;

void* processTile(void* arg) {
  while (1) {
    pthread_mutex_lock(&lock);
    int tile = nextTile++;
    pthread_mutex_unlock(&lock);

    if (tile >= (N/TILE)*(N/TILE))
        break;

    int row = (tile / (N/TILE)) * TILE;
    int col = (tile % (N/TILE)) * TILE;

    double local_min = 1000000000;
```

```
double local_max = -1000000000;
double sum = 0, sq_sum = 0;
int count = 0;

for (int i = row; i < row + TILE; i++) {
    for (int j = col; j < col + TILE; j++) {
        double val = globalMatrix[i][j];
        if (val < local_min) local_min = val;
        if (val > local_max) local_max = val;
        sum += val;
        sq_sum += val * val;
        count++;
    }
}

double mean = sum / count;
double variance = (sq_sum / count) - (mean * mean);
double std_dev = sqrt(variance);

for (int i = row; i < row + TILE; i++) {
    for (int j = col; j < col + TILE; j++) {
        if (fabs(globalMatrix[i][j] - mean) > 2 * std_dev)
            anomaly_count++;
```

```
            }
        }

        pthread_mutex_lock(&lock);
        if (local_min < global_min) global_min = local_min;
        if (local_max > global_max) global_max = local_max;
        global_sum += sum;
        pthread_mutex_unlock(&lock);
    }
    pthread_exit(NULL);
}

void* hotspotTask(void* arg) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            if (globalMatrix[i][j] > T_HOT) {
                pthread_mutex_lock(&lock);
                hotspot_count++;
                pthread_mutex_unlock(&lock);
            }
        }
    }
    pthread_exit(NULL);
```

```
}

void* coldspotTask(void* arg) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            if (globalMatrix[i][j] < T_COLD) {
                pthread_mutex_lock(&lock);
                coldspot_count++;
                pthread_mutex_unlock(&lock);
            }
        }
    }
    pthread_exit(NULL);
}

double normalized[N][N];

void* normalizeTask(void* arg) {
    double range = global_max - global_min;
    if (range == 0) range = 1;
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            normalized[i][j] =
```

```
            (globalMatrix[i][j] - global_min) / range;
        }
    }
    pthread_exit(NULL);
}

double risk[N][N];

void computeRisk() {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            double proximity_hot = 1;
            double proximity_cold = 1;
            risk[i][j] =
                normalized[i][j] * proximity_hot /
                (proximity_cold + 1);
        }
    }
}

int main() {
    initializeData();
```

```
pthread_t satThreads[5];
int ids[5];

for (int i = 0; i < 5; i++) {
    ids[i] = i;
    pthread_create(&satThreads[i], NULL, processSatellite, &ids[i]);
}

for (int i = 0; i < 5; i++)
    pthread_join(satThreads[i], NULL);

mergeMatrices(5);

pthread_t tileThreads[MAX_THREADS];
for (int i = 0; i < MAX_THREADS; i++)
    pthread_create(&tileThreads[i], NULL, processTile, NULL);

for (int i = 0; i < MAX_THREADS; i++)
    pthread_join(tileThreads[i], NULL);

pthread_t tA, tB, tC;
pthread_create(&tA, NULL, hotspotTask, NULL);
pthread_create(&tB, NULL, coldspotTask, NULL);
```

```
    pthread_create(&tC, NULL, normalizeTask, NULL);

    pthread_join(tA, NULL);
    pthread_join(tB, NULL);
    pthread_join(tC, NULL);

    computeRisk();

    printf("Max: %f\n", global_max);
    printf("Min: %f\n", global_min);
    printf("Hotspots: %d\n", hotspot_count);
    printf("Coldspots: %d\n", coldspot_count);

    return 0;
}
```

QS3.

```
system@system:~/Desktop/Assignment_2$ gedit question3.c
system@system:~/Desktop/Assignment_2$ gcc question3.c -o question3 -lpthread
system@system:~/Desktop/Assignment_2$ ./question3
Patient 0 arrived Type: 2
Doctor 1 treating Patient 0 Type: 2
Patient 1 arrived Type: 1
Doctor 2 treating Patient 1 Type: 1
Patient 2 arrived Type: 1
Patient 3 arrived Type: 2
Patient 4 arrived Type: 2
Patient 5 arrived Type: 2
Patient 6 arrived Type: 1
Patient 7 arrived Type: 0
Patient 8 arrived Type: 2
Patient 9 arrived Type: 1
Doctor 1 treating Patient 3 Type: 2
Patient 10 arrived Type: 2
Doctor 2 treating Patient 2 Type: 1
Patient 11 arrived Type: 2
Patient 12 arrived Type: 1
Patient 13 arrived Type: 2
Patient 14 arrived Type: 2
```

```
Doctor 2 treating Patient 12 Type: 1
Doctor 1 treating Patient 10 Type: 2
Doctor 2 treating Patient 16 Type: 1
Doctor 1 treating Patient 11 Type: 2
Doctor 2 treating Patient 17 Type: 1
Doctor 1 treating Patient 13 Type: 2
Doctor 2 treating Patient 18 Type: 1
Doctor 1 treating Patient 14 Type: 2
Doctor 2 treating Patient 19 Type: 1
Doctor 1 treating Patient 15 Type: 2
Doctor 2 treating Patient 24 Type: 1
Doctor 1 treating Patient 21 Type: 2
Doctor 2 treating Patient 26 Type: 1
Doctor 1 treating Patient 27 Type: 2
Doctor 2 treating Patient 28 Type: 1
Doctor 1 treating Patient 7 Type: 0
Doctor 2 treating Patient 20 Type: 0
Doctor 1 treating Patient 22 Type: 0
Doctor 2 treating Patient 23 Type: 0
Doctor 1 treating Patient 25 Type: 0
Doctor 2 treating Patient 29 Type: 0
All patients processed
```

CODE:

```
#include <pthread.h>
#include <unistd.h>
#include <stdio.h>
#include <time.h>
#include <stdlib.h>

#define MAX 200
#define TOTAL_PATIENTS 30

typedef struct {
    int id;
    int type;
    time_t arrival_time;
} Patient;

Patient criticalQ[MAX], seriousQ[MAX], normalQ[MAX];
int c_front = 0, c_rear = 0;
int s_front = 0, s_rear = 0;
int n_front = 0, n_rear = 0;

int treated_count = 0;
```

```
pthread_mutex_t mutexQ = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t cond_doctor = PTHREAD_COND_INITIALIZER;

typedef struct {
    int id;
    int isSenior;
    int normalCount;
} Doctor;

void enqueue(Patient arr[], int *rear, Patient p) {
    arr[(*rear)++] = p;
}

Patient dequeue(Patient arr[], int *front) {
    return arr[(*front)++];
}

int isEmpty(int front, int rear) {
    return front == rear;
}

void* patientThread(void* arg) {
    Patient p = *(Patient*)arg;
```

```
    pthread_mutex_lock(&mutexQ);

    p.arrival_time  = time(NULL);

    if (p.type == 2)
        enqueue(criticalQ,  &c_rear, p);
    else if (p.type == 1)
        enqueue(seriousQ,  &s_rear, p);
    else
        enqueue(normalQ,  &n_rear, p);

    printf("Patient  %d arrived  Type: %d\n", p.id, p.type);

    pthread_cond_signal(&cond_doctor);
    pthread_mutex_unlock(&mutexQ);

    pthread_exit(NULL);
}

void* doctorThread(void*  arg) {
    Doctor* doc = (Doctor*)arg;
```

```
    while (1) {
        pthread_mutex_lock(&mutexQ);

        if (treated_count >= TOTAL_PATIENTS) {
            pthread_mutex_unlock(&mutexQ);
            break;
        }

        while (isEmpty(c_front, c_rear) && isEmpty(s_front, s_rear) && isEmpty(n_front, n_rear))
{
            pthread_cond_wait(&cond_doctor, &mutexQ);
        }

        Patient p;
        int found = 0;

        if (!isEmpty(c_front, c_rear) && doc->isSenior) {
            p = dequeue(criticalQ, &c_front);
            found = 1;
        }
        else if (!isEmpty(s_front, s_rear)) {
            p = dequeue(seriousQ, &s_front);
            found = 1;
        }
```

```
else if (!isEmpty(n_front, n_rear)) {
    p = dequeue(normalQ, &n_front);
    found = 1;
}

if (!found) {
    pthread_mutex_unlock(&mutexQ);
    continue;
}

if (p.type == 0)
    doc->normalCount++;
else
    doc->normalCount = 0;

if (doc->normalCount == 3 && !isEmpty(s_front, s_rear)) {
    p = dequeue(seriousQ, &s_front);
    doc->normalCount = 0;
}

treated_count++;

pthread_mutex_unlock(&mutexQ);
```

```
        printf("Doctor %d treating Patient %d Type: %d\n", doc->id, p.id, p.type);

        sleep(1);
    }

    pthread_exit(NULL);
}

int main() {
    srand(time(NULL));

    pthread_t docThreads[2];
    Doctor docs[2] = {
        {1, 1, 0}, // Senior
        {2, 0, 0}  // Junior
    };

    for (int i = 0; i < 2; i++)
        pthread_create(&docThreads[i], NULL, doctorThread, &docs[i]);

    pthread_t patients[TOTAL_PATIENTS];
    Patient pts[TOTAL_PATIENTS];
```

```
    for (int i = 0; i < TOTAL_PATIENTS; i++) {
        pts[i].id = i;
        pts[i].type = rand() % 3;
        pts[i].arrival_time = time(NULL);
        pthread_create(&patients[i], NULL, patientThread, &pts[i]);
        usleep(100000);
    }

    for (int i = 0; i < TOTAL_PATIENTS; i++)
        pthread_join(patients[i], NULL);

    pthread_cond_broadcast(&cond_doctor);

    for (int i = 0; i < 2; i++)
        pthread_join(docThreads[i], NULL);

    printf("All patients processed\n");

    return 0;
}
```

QS4.

```
All patients processed
system@system:~/Desktop/Assignment_2$ gedit question4.c
system@system:~/Desktop/Assignment_2$ gcc question4.c -o question4 -lpthread
system@system:~/Desktop/Assignment_2$ ./question4
Generated Flight 1 Type 1
Runway 0 handling Flight 1 Type 1
Generated Flight 2 Type 4
Runway 1 handling Flight 2 Type 4
Generated Flight 3 Type 4
Generated Flight 4 Type 4
Generated Flight 5 Type 3
Generated Flight 6 Type 4
Runway 0 handling Flight 3 Type 4
Runway 1 handling Flight 4 Type 4
Generated Flight 7 Type 3
Generated Flight 8 Type 4
Generated Flight 9 Type 2
Generated Flight 10 Type 4
Runway 0 handling Flight 6 Type 4
Generated Flight 11 Type 4
Runway 1 handling Flight 8 Type 4
Generated Flight 12 Type 2
Generated Flight 13 Type 2
```

```
Runway 0 handling Flight 10 Type 4
Generated Flight 16 Type 2
Runway 1 handling Flight 11 Type 4
Generated Flight 17 Type 1
Generated Flight 18 Type 3
Generated Flight 19 Type 2
Generated Flight 20 Type 3
Runway 0 handling Flight 7 Type 3
Runway 1 handling Flight 5 Type 3
Runway 0 handling Flight 14 Type 3
Runway 1 handling Flight 15 Type 3
Runway 0 handling Flight 18 Type 3
Runway 1 handling Flight 20 Type 3
Runway 0 handling Flight 12 Type 2
Runway 1 handling Flight 9 Type 2
Runway 0 handling Flight 13 Type 2
Runway 1 handling Flight 16 Type 2
Runway 0 handling Flight 19 Type 2
Runway 1 handling Flight 17 Type 1
All flights handled: 20
system@system:~/Desktop/Assignment_2$ echo 'Muhammad Adil Saeed 24k-0705'
Muhammad Adil Saeed 24k-0705
system@system:~/Desktop/Assignment_2$
```

CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
#include <time.h>

#define MAX_FLIGHTS 100
#define RUNWAYS 2
#define TOTAL_FLIGHTS 20

#define TAKEOFF 1
#define LANDING 2
#define LOW_FUEL 3
#define EMERGENCY 4

typedef struct {
    int id;
    int type;
    int priority;
    time_t arrivalTime;
} Flight;
```

```
Flight flightQueue[MAX_FLIGHTS];
int flightCount = 0;

int handledFlights = 0;
int done = 0;

pthread_mutex_t queueLock;
pthread_mutex_t runwayLock[RUNWAYS];
pthread_cond_t queueCond;

void sortQueue() {
    for (int i = 0; i < flightCount - 1; i++) {
        for (int j = i + 1; j < flightCount; j++) {
            if (flightQueue[i].priority < flightQueue[j].priority) {
                Flight t = flightQueue[i];
                flightQueue[i] = flightQueue[j];
                flightQueue[j] = t;
            }
        }
    }
}

void insertFlight(Flight f) {
```

```
    flightQueue[flightCount++] = f;
    sortQueue();
}

Flight getFlight() {
    Flight f = flightQueue[0];
    for (int i = 1; i < flightCount; i++)
        flightQueue[i - 1] = flightQueue[i];
    flightCount--;
    return f;
}

void updatePriority() {
    time_t now = time(NULL);
    for (int i = 0; i < flightCount; i++) {
        if (difftime(now, flightQueue[i].arrivalTime) > 5)
            flightQueue[i].priority++;
    }
    sortQueue();
}

void* generator(void* arg) {
    for (int i = 1; i <= TOTAL_FLIGHTS; i++) {
```

```
        Flight f;
        f.id = i;
        f.type = rand() % 4 + 1;
        f.priority = f.type;
        f.arrivalTime = time(NULL);

        pthread_mutex_lock(&queueLock);
        insertFlight(f);
        printf("Generated Flight %d Type %d\n", f.id, f.type);
        pthread_cond_signal(&queueCond);
        pthread_mutex_unlock(&queueLock);

        usleep(200000);
    }

    pthread_mutex_lock(&queueLock);
    done = 1;
    pthread_cond_broadcast(&queueCond);
    pthread_mutex_unlock(&queueLock);

    return NULL;
}
```

```
void* runway(void* arg) {
    int id = *(int*)arg;

    while (1) {
        pthread_mutex_lock(&queueLock);

        while (flightCount == 0 && !done)
            pthread_cond_wait(&queueCond, &queueLock);

        if (flightCount == 0 && done) {
            pthread_mutex_unlock(&queueLock);
            break;
        }

        updatePriority();
        Flight f = getFlight();
        pthread_mutex_unlock(&queueLock);

        pthread_mutex_lock(&runwayLock[id]);
        printf("Runway %d handling Flight %d Type %d\n", id, f.id, f.type);
        sleep(1);
        pthread_mutex_unlock(&runwayLock[id]);
```

```
        pthread_mutex_lock(&queueLock);
        handledFlights++;
        pthread_mutex_unlock(&queueLock);
    }

    return NULL;
}

int main() {
    srand(time(NULL));

    pthread_mutex_init(&queueLock, NULL);
    pthread_cond_init(&queueCond, NULL);

    for (int i = 0; i < RUNWAYS; i++)
        pthread_mutex_init(&runwayLock[i], NULL);

    pthread_t gen;
    pthread_create(&gen, NULL, generator, NULL);

    pthread_t r[RUNWAYS];
    int ids[RUNWAYS];
```

```
    for (int i = 0; i < RUNWAYS; i++) {
        ids[i] = i;
        pthread_create(&r[i], NULL, runway, &ids[i]);
    }

    pthread_join(gen, NULL);

    for (int i = 0; i < RUNWAYS; i++)
        pthread_join(r[i], NULL);

    printf("All flights handled: %d\n", handledFlights);

    return 0;
}
```

Question 2:

Task Parallelism
- Satellite preprocessing and hotspot, coldspot and normalization thread.

Data Parallelism
Matrix divided into tiles and each thread processes a tile independently.

Dynamic Load Balancing
- Shared next tile variable.
- Threads pick work dynamically.

Synchronization.
Mutex used for
global stats updates and counters.

Interdependency handling:
pthread_join() ensures risk calculation waits for normilization and detection.

Question 3:

* Data Structures
- Three queues, a struct tracking is_sensor;

* Deadlock prevention.
- Single mutex hierarchy used
All threads must acquire the single mutex before accessing shared state. Avoid used of nested mutex locks which usually cause deadlock.

Question 4:
- Data structure:
1) Priority queue used here which is a sorted array of flight structs prioritized by integer score.
2) Integer array for runway status and pthread_cond_t used to make controllers only when a flight is added or cleared.

Question 5:

The paper "parallel processing of E-Aflex algorithm using pthread paradigm" addressed the challenge of searching through massive databases by parallelizing the E-Aflex string matching algorithm using POSIX threads. Traditionally, searching for specific pattern in large dataset like DNA sequences is slow because it happens sequentially. However, authors propose a divide and conquer approach where database is split into segments processed simultaneously across multiple CPU cores. By testing the algorithm on various data-types, the research demonstrate that this multi-threaded approach significantly reduces execution time and provides speedup compared to original version. It concludes that adding threads improves efficiency and that there is a limit where communication over head between cores begins to diminish returns, highlighting a balance between resources and algorithmic performances.